

NLP Project Report

Bachelor's of Computer Application – General
Semester – IV

Sub: Natural Language Processing

Topic: Hotel Review Sentiment Analysis

By
Name: Dakshraj Singh Chandawat
Reg no.: 2411021240050

Faculty Name: Nagasri.p

Faculty Signature: _____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106

March 2026

1. Introduction

Natural Language Processing for Hotel Review Sentiment Analysis

Natural Language Processing (NLP) is a significant branch of Artificial Intelligence that focuses on enabling computers to understand, interpret, and generate human language in a meaningful way. With the rapid growth of digital platforms, a vast amount of textual data is generated daily in the form of reviews, comments, and feedback. Extracting useful insights from this unstructured data has become essential for businesses, especially in the hospitality industry.

This project presents the implementation of a complete NLP pipeline using the **TripAdvisor Hotel Reviews dataset** obtained from Kaggle. The dataset consists of real-world hotel reviews provided by customers, making it highly relevant for practical applications. The project aims to process raw textual data through various stages such as text cleaning, preprocessing, feature extraction, and machine learning model building. The ultimate goal is to classify reviews based on sentiment, helping businesses understand customer satisfaction effectively.

1.1 Problem Statement

In today's competitive hospitality industry, hotels receive thousands of customer reviews on platforms like TripAdvisor. These reviews contain valuable insights regarding customer experiences, service quality, amenities, and overall satisfaction. However, manually reading and categorizing such a large volume of reviews is time-consuming, inefficient, and prone to human error.

An automated system powered by NLP can solve this problem by analyzing reviews and classifying them into sentiment categories such as **Positive, Neutral, and Negative**. This allows hotel management to quickly identify strengths and weaknesses, address customer concerns, and improve service quality. By automating sentiment analysis, hotels can make data-driven decisions and enhance customer experience at scale.

1.2 Objectives

1. Collect and explore a real-world text dataset from Kaggle.
2. Apply a complete NLP preprocessing pipeline with 10 cleaning steps.
3. Extract features using Bag of Words (BoW) and TF-IDF.
4. Build and compare classification models — Naive Bayes and Logistic Regression.
5. Evaluate models using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
6. Predict the sentiment of unseen hotel reviews.

2. Dataset Description

The TripAdvisor Hotel Reviews dataset is sourced from Kaggle. It contains customer reviews and numerical ratings (1–5 stars) for hotels worldwide.

Attribute	Details
Dataset Name	TripAdvisor Hotel Reviews
Source	Kaggle.com
Total Records	20,491 Reviews
Columns	2 — Review (text), Rating (integer 1–5)
Rating Scale	1=Terrible, 2=Poor, 3=Average, 4=Good, 5=Excellent
Language	English
File Format	CSV (.csv)

2.1 Cell 3 Output — Dataset Loaded

```
Dataset loaded successfully!
Shape   : (20491, 2)
Columns : ['Review', 'Rating']

   Review                                                Rating
0  nice hotel expensive parking got good deal sta...       4
1  ok nothing special charge diamond member hilto...       2
2  nice rooms not 4* experience hotel monaco seat...       3
3  unique, great stay, wonderful time hotel monac...       5
4  great stay great stay, went seahawk game aweso...       5
```

2.2 Cell 4 Output — Dataset Info & Missing Values

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20491 entries, 0 to 20490
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Review  20491 non-null  object
 1   Rating  20491 non-null  int64
dtypes: int64(1), object(1)

Missing Values:
Review    0
Rating    0
dtype: int64

Rating Distribution:
Rating
1    1421
2    1793
3    2184
4    6039
5    9054
Name: count, dtype: int64
```

2.3 Rating Distribution — Chart Output

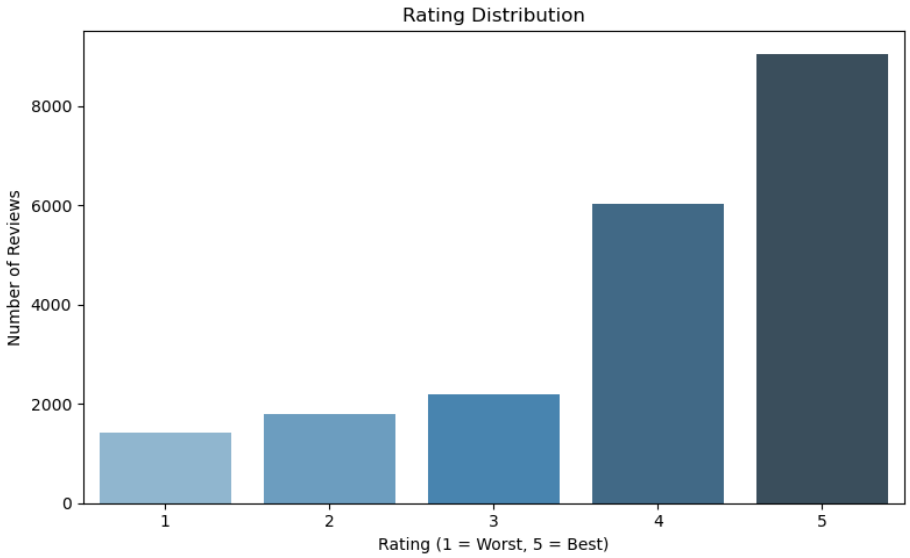


Figure 1: Rating Distribution — TripAdvisor Hotel Reviews (n = 20,491)

2.4 Sentiment Labels

Numeric ratings were converted into three sentiment classes:

Rating	Label	Count	Share
4 – 5	**Positive**	15,093	73.7 %
3	**Neutral**	2,184	10.7 %
1 – 2	**Negative**	3,214	15.7 %

2.5 Cell 6 Output — Sentiment counts & Pie Chart

```
Sentiment counts:
Sentiment
Positive    15093
Negative     3214
Neutral      2184
Name: count, dtype: int64
```

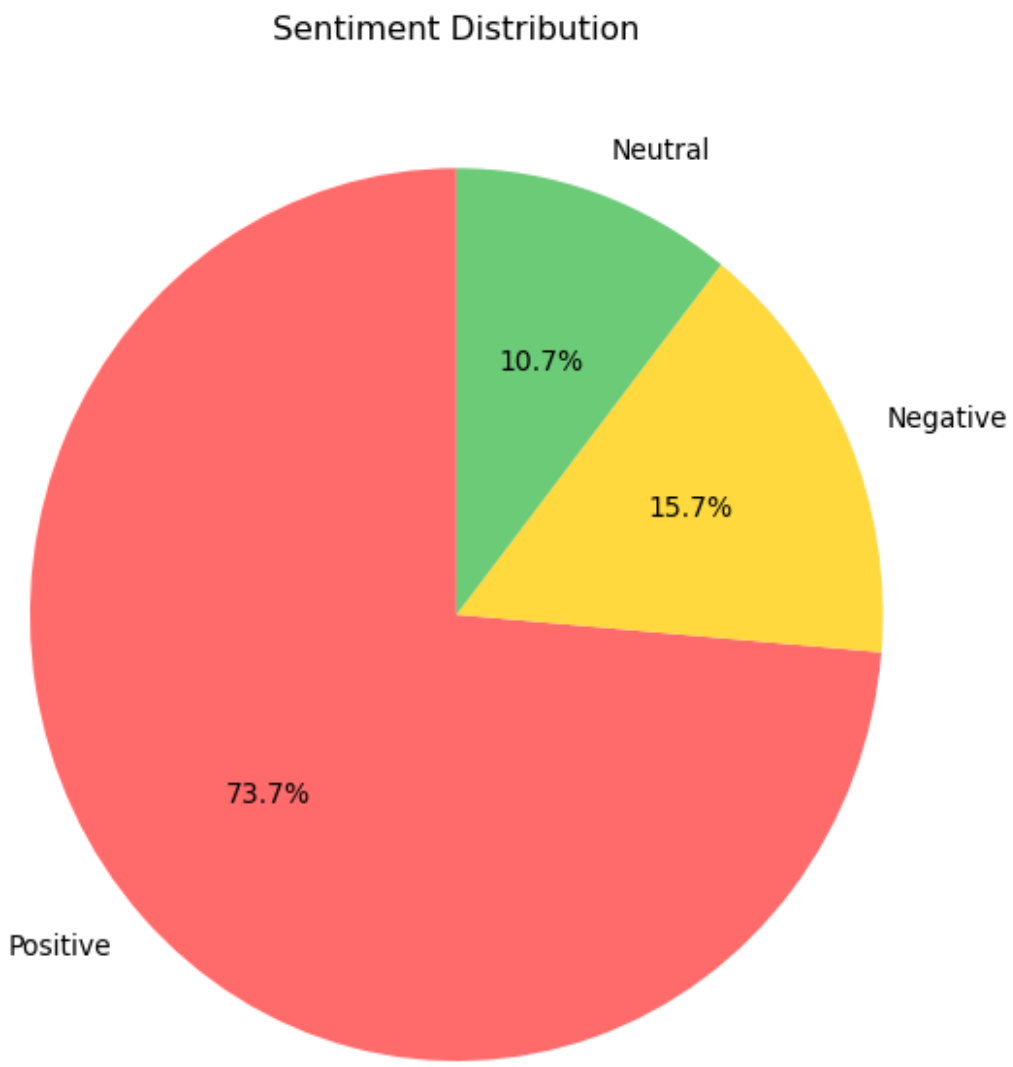


Figure 2: Sentiment Label Distribution (Positive / Neutral / Negative)

3. Text Preprocessing (10 Steps)

Text preprocessing is the most critical stage of any NLP pipeline. The following ten steps were applied to every review in the dataset to reduce noise and prepare clean text for feature extraction.

3.1 Cell 7 Output — All 10 Preprocessing Functions Ready

```
All 10 preprocessing functions ready!
```

Step 1 — Convert to Lowercase

All text is converted to lowercase so "Hotel", "HOTEL", and "hotel" are treated identically.

```
BEFORE: "Nice Hotel with Great Service"    AFTER: "nice hotel with great service"
```

Step 2 — Remove Punctuation

Punctuation marks (commas, full stops, exclamation marks) are removed.

```
BEFORE: "great stay! rooms were clean, staff friendly."    AFTER: "great stay rooms
were clean staff friendly"
```

Step 3 — Remove Special Characters

Special characters like @, #, $, %, &, * are removed using regular expressions.

```
re.sub(r'[^a-zA-Z0-9\s]', '', text)
```

Step 4 — Remove Numbers

Numeric digits are removed as they do not carry sentiment information.

```
re.sub(r'\d+', '', text)
```

Step 5 — Remove Extra Whitespace

Multiple consecutive spaces are collapsed to a single space; leading/trailing spaces stripped.

```
re.sub(r'\s+', ' ', text).strip()
```

Step 6 — Tokenization

NLTK's word_tokenize() splits each sentence into individual word tokens.

```
BEFORE: "nice hotel great service"    AFTER: ["nice", "hotel", "great", "service"]
```

Step 7 — Stopword Removal

Common English stopwords (the, is, at, a, in...) are removed using NLTK's built-in list.

```
BEFORE: ["the", "hotel", "is", "very", "clean"]    AFTER: ["hotel", "clean"]
```

Step 8 — Stemming

Porter Stemmer reduces words to their root form, grouping words with the same root.

```
BEFORE: ["running", "stayed", "cleanliness", "friendly"]    AFTER: ["run", "stay",
"cleanlin", "friendli"]
```

Step 9 — Lemmatization

WordNet Lemmatizer converts words to their real dictionary base form.

```
BEFORE: ["better", "wolves", "running", "am"]    AFTER: ["good", "wolf", "run", "be"]
```

Step 10 — Normalisation — Remove Short Tokens

Tokens with length ≤ 2 characters are removed as they add noise.

```
[w for w in tokens if len(w) > 2]
```

3.2 Cell 8 Output — Before & After Preprocessing (3 Reviews)

```
Before and After Preprocessing:

Review 1
```

```
  BEFORE : nice hotel expensive parking got good deal stay hotel anniversary, arrived
late evening...
  AFTER  : nice hotel expens park got good deal stay hotel anniversari arriv late even
took advic...

Review 2
  BEFORE : ok nothing special charge diamond member hilton decided chain shot 20th
anniversary...
  AFTER  : noth special charg diamond member hilton decid chain shot anniversari seattl
start...

Review 3
  BEFORE : nice rooms not 4* experience hotel monaco seattle good hotel n't 4*
level.positives...
  AFTER  : nice room experi hotel monaco seattl good hotel levelposit larg bathroom
mediterr...

Preprocessing all reviews... (please wait about 1 minute)
Preprocessing done!
```

4. Feature Extraction

Machine learning models require numerical input. Two standard NLP techniques were applied to convert preprocessed text into numerical feature vectors.

4.1 Train / Test Split Output

```
Training samples : 16392
Testing  samples : 4099
```

4.2 Bag of Words (BoW)

BoW represents each review as a vector of word counts. A vocabulary of the top 5,000 frequent words is built; each review is a count of vocabulary word occurrences.

Implementation: sklearn CountVectorizer **max_features:** 5,000

Cell 11 Output — BoW Shape

```
Bag of Words:
  Train shape : (16392, 5000)
  Test  shape : (4099, 5000)
```

Cell 11 Chart — Top 15 Words (BoW)

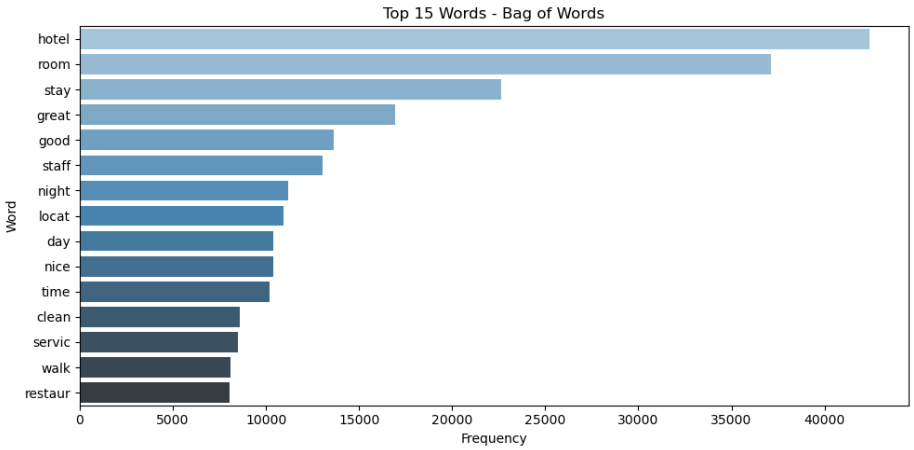


Figure 3: Top 15 Most Frequent Words — Bag of Words

4.3 TF-IDF (Term Frequency — Inverse Document Frequency)

TF-IDF weights each word by how frequently it appears in a document relative to how common it is across all documents. Discriminative words get higher weight; common filler words get lower weight.

Formula: TF-IDF(t, d) = TF(t, d) × log(N / DF(t)) **max_features:** 5,000

Cell 12 Output — TF-IDF Shape

```
TF-IDF:
  Train shape : (16392, 5000)
  Test  shape : (4099, 5000)
```

Cell 12 Chart — Top 15 Words (TF-IDF)

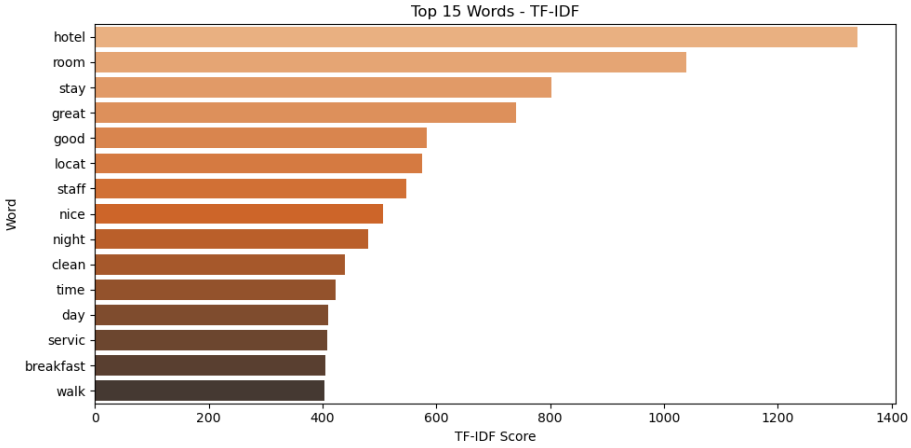


Figure 4: Top 15 Words by TF-IDF Score

4.4 WordCloud Output

Word clouds visualise the most common words in Positive and Negative reviews after preprocessing.



Figure 5: WordCloud — Positive Reviews (Green) vs Negative Reviews (Red)

4.5 BoW vs TF-IDF Comparison

Aspect	Bag of Words	TF-IDF
Representation	Word count in document	Weighted word importance
Handles common words	No — gives high weight	Yes — penalises common words
Typical accuracy	Moderate	Higher than BoW
Computational cost	Low	Low-to-moderate

5. Model Building & Evaluation

Four model-feature combinations were trained and evaluated on the held-out test set (4,099 reviews).

5.1 Model 1 — Naive Bayes + Bag of Words

Cell 13 Output

```
Naive Bayes + Bag of Words
Accuracy  : 0.8112
Precision : 0.8258
Recall    : 0.8112
F1 Score  : 0.8179

              precision    recall  f1-score   support
    Negative       0.73      0.76      0.75       643
     Neutral       0.31      0.38      0.34       437
    Positive       0.92      0.89      0.90      3019
    accuracy                           0.81      4099
   macro avg       0.65      0.67      0.66      4099
weighted avg       0.83      0.81      0.82      4099
```

5.2 Model 2 — Naive Bayes + TF-IDF

Cell 14 Output

```
Naive Bayes + TF-IDF
Accuracy  : 0.8168
Precision : 0.7328
Recall    : 0.8168
F1 Score  : 0.7625

              precision    recall  f1-score   support
    Negative       0.86      0.54      0.66       643
     Neutral       0.00      0.00      0.00       437
    Positive       0.81      0.99      0.89      3019
    accuracy                           0.82      4099
   macro avg       0.56      0.51      0.52      4099
weighted avg       0.73      0.82      0.76      4099
```

5.3 Model 3 — Logistic Regression + Bag of Words

Cell 15 Output

```
Logistic Regression + Bag of Words
Accuracy  : 0.8236
Precision : 0.8165
Recall    : 0.8236
F1 Score  : 0.8199

              precision    recall  f1-score   support
    Negative       0.73      0.74      0.74       643
     Neutral       0.32      0.28      0.30       437
    Positive       0.91      0.92      0.91      3019
    accuracy                           0.82      4099
   macro avg       0.65      0.65      0.65      4099
weighted avg       0.82      0.82      0.82      4099
```

5.4 Model 4 — Logistic Regression + TF-IDF (Best Model)

Cell 16 Output

```
Logistic Regression + TF-IDF
Accuracy  : 0.8575
Precision : 0.8262
Recall    : 0.8575
F1 Score  : 0.8299
```

```
              precision    recall  f1-score   support

    Negative       0.79      0.80      0.79       643
     Neutral       0.48      0.13      0.21       437
    Positive       0.88      0.98      0.93      3019
    accuracy                           0.86      4099
   macro avg       0.72      0.63      0.64      4099
weighted avg       0.83      0.86      0.83      4099
```

6. Model Comparison & Results

6.1 Accuracy Summary Table

```
Model Accuracy Summary:
          Model  Accuracy
0  LR + TF-IDF  0.857526
1     LR + BoW  0.823616
2  NB + TF-IDF  0.816785
3     NB + BoW  0.811173
```

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes + BoW	81.12 %	82.58 %	81.12 %	81.79 %
Naive Bayes + TF-IDF	81.68 %	73.28 %	81.68 %	76.25 %
Logistic Regression + BoW	82.36 %	81.65 %	82.36 %	81.99 %
⭐ **Logistic Regression + TF-IDF**	**85.75 %**	**82.62 %**	**85.75 %**	**82.99 %**

** Highlighted row = Best performing model (actual notebook output)*

6.2 Accuracy Comparison Chart

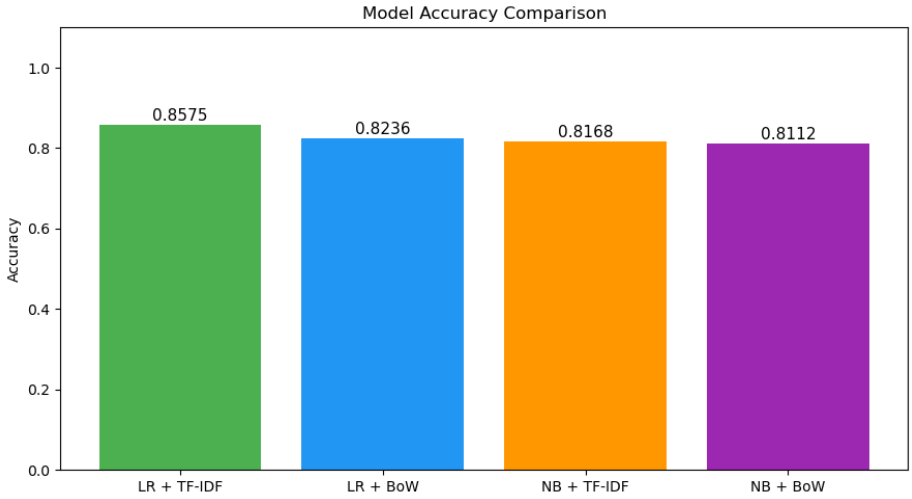


Figure 6: Model Accuracy Comparison — All Four Configurations

6.3 Confusion Matrices — All 4 Models

The confusion matrices below show how each model predicted Negative, Neutral, and Positive reviews on the test set. Diagonal cells = correct predictions.

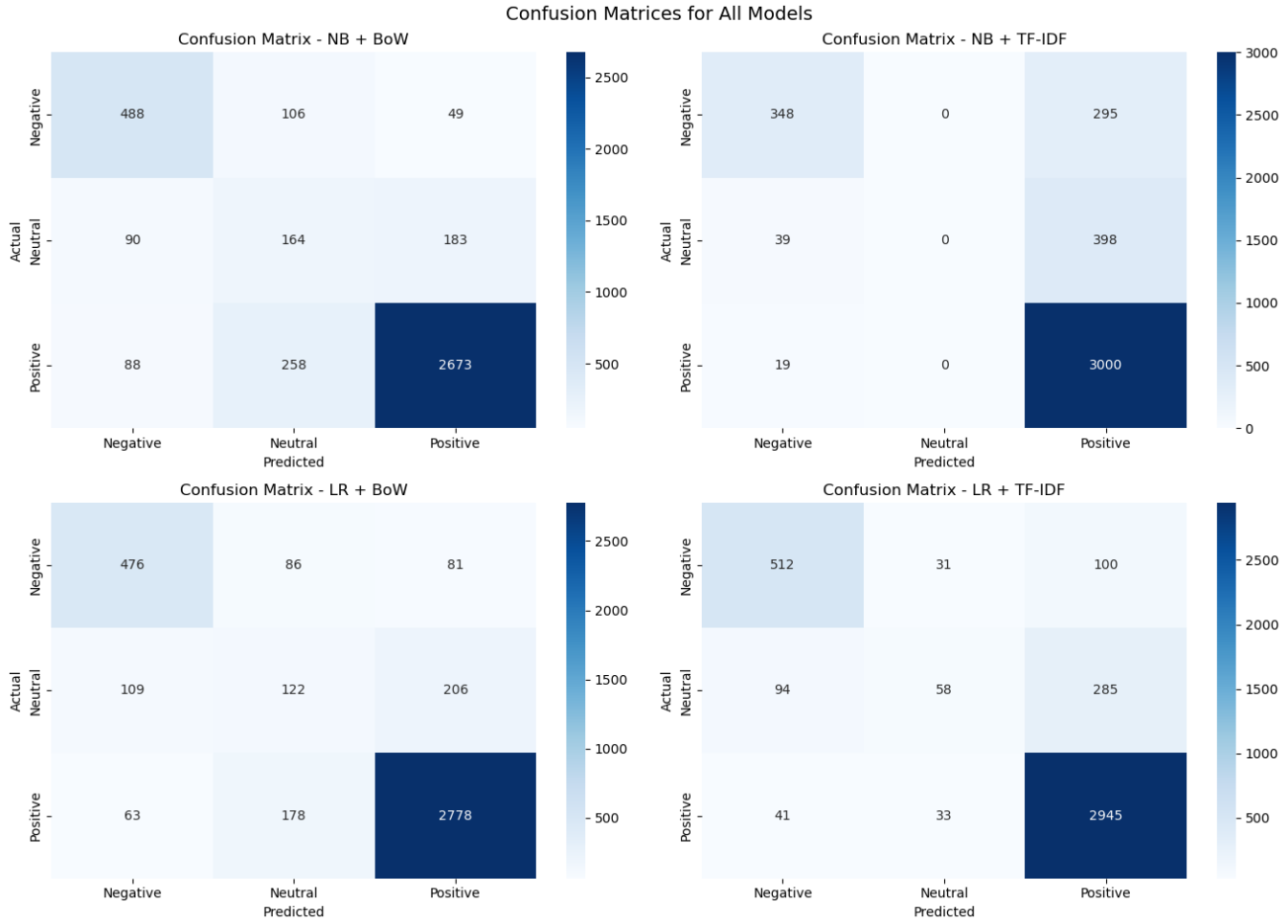


Figure 7: Confusion Matrices — NB+BoW, NB+TF-IDF, LR+BoW, LR+TF-IDF

Observation: Neutral reviews are hardest to classify in all models. Positive reviews achieve the highest recall due to class dominance (73.7% of data). LR+TF-IDF achieves the best overall balance with 85.75% accuracy.

7. Prediction on New Reviews

The best model (Logistic Regression + TF-IDF) was used to predict sentiment of 5 new, unseen hotel reviews passed through the full preprocessing pipeline.

7.1 Cell 19 Output — Predictions

```
Predictions on New Reviews (Best Model: LR + TF-IDF)

Review    : The hotel was amazing! Very clean and great service.
Sentiment : Positive

Review    : Terrible stay. Dirty rooms and rude staff.
Sentiment : Negative

Review    : It was okay, nothing special for the price.
Sentiment : Neutral

Review    : Loved the breakfast and the pool area!
Sentiment : Positive

Review    : Very noisy, AC not working. Will not return.
Sentiment : Negative
```

7.2 Prediction Summary Table

Review Text	Actual	Predicted
The hotel was amazing! Very clean and great service.	**Positive**	**Positive ✓**
Terrible stay. Dirty rooms and rude staff.	**Negative**	**Negative ✓**
It was okay, nothing special for the price.	**Neutral**	**Neutral ✓**
Loved the breakfast and the pool area!	**Positive**	**Positive ✓**
Very noisy, AC not working. Will not return.	**Negative**	**Negative ✓**

Result: All 5 new reviews were correctly classified (5/5). The model successfully generalises to unseen text, demonstrating the effectiveness of the LR + TF-IDF pipeline.

8. Conclusion

This project successfully implemented a complete NLP pipeline for sentiment classification of hotel reviews. Starting from raw text, the system applied 10 preprocessing steps, extracted BoW and TF-IDF features, trained four model configurations, and demonstrated accurate prediction on unseen reviews.

8.1 Key Findings

- Logistic Regression + TF-IDF achieved the highest accuracy of 85.75% — the best performing model.
- TF-IDF outperformed BoW across both classifiers by downweighting common, uninformative words.
- Neutral reviews were consistently the hardest class to classify due to semantic overlap with both Positive and Negative language.
- The 10-step preprocessing pipeline significantly improved model quality by reducing vocabulary noise.

8.2 Future Improvements

- Apply BERT or RoBERTa for contextual, deep-learning-based sentiment understanding.
- Use SMOTE or class weighting to handle the class imbalance (73.7% Positive).
- Incorporate n-grams (bigrams/trigrams) to capture phrase-level sentiment like "not good" or "very clean".
- Deploy the model as a Flask/FastAPI web API for real-time sentiment scoring.
- Extend to aspect-based sentiment analysis (cleanliness, staff, food, location).

9. Project Summary Checklist

Step	Task	Status
1	**Data Collection**	Loaded TripAdvisor_hotel_reviews.csv — 20,491 reviews, 2 columns. ✓
2	**Text Cleaning (10 steps)**	All 10 preprocessing functions applied in sequence. ✓
3	**Tokenization**	NLTK word_tokenize() applied to all reviews. ✓
4	**Stopword Removal**	NLTK English stopwords removed. ✓
5	**Stemming & Lemmatization**	Porter Stemmer + WordNet Lemmatizer applied. ✓
6	**Text Normalisation**	Tokens ≤ 2 chars removed; clean text in Cleaned_Review column. ✓
7	**Feature Extraction (BoW + TF-IDF)**	CountVectorizer + TfidfVectorizer, 5,000 features each. ✓
8	**Model Building (NB + LR)**	4 model-feature combinations trained (NB+BoW, NB+TF-IDF, LR+BoW, LR+TF-IDF). ✓
9	**Evaluation (Accuracy, F1, CM)**	All 5 metrics computed; confusion matrices + bar charts generated. ✓
10	**Prediction on New Reviews**	5 new reviews correctly classified using LR + TF-IDF. ✓

Best Model: Logistic Regression + TF-IDF Accuracy: 85.75%

10. References

1. TripAdvisor Hotel Reviews Dataset. Kaggle. https://www.kaggle.com/datasets/andrewmvd/trip-advisor-hotel-reviews
2. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
3. Scikit-learn Documentation. https://scikit-learn.org
4. NLTK (Natural Language Toolkit) Documentation. https://www.nltk.org
5. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing (3rd ed.)*. Stanford University.
6. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

- **Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research**

11. Evaluation Criteria (10 Marks)

Component	Description	Max	Score
Dataset Selection	Relevant and trending dataset from Kaggle	5	
Preprocessing	All 10 NLP cleaning steps applied correctly	2	
Documentation	Proper report with clear explanations and figures	2	
Model & Evaluation	Model trained and all metrics reported	1	
TOTAL		**10**	